How Does AI Remember You in a New Chat?

You open a new chat, ask for dinner ideas, and get veggie tacos. If the app has saved your vegetarian preference, it can make that information available for this reply.

Think of the model's current context as the notes on a desk. The app assembles those notes before the model answers. Saved memory is like a drawer the app can pull from.

This guide uses an illustrative memory design. Apps differ in what they retain, when they retrieve it, and how users control it.

In the same chat, the app may pass earlier turns alongside your new message. The model can use those turns while they remain available. The context window is a token budget. Depending on the app, older turns may be omitted or summarised when the conversation grows.

A summary can retain the main topic and drop a small detail. For example, "We discussed dinner" preserves the topic while losing "no mushrooms."

Save, retrieve, use

1. Save	2. Retrieve	3. Use

Save. The app stores a useful preference, such as “prefers vegetarian meals.” A memory entry can carry a date and source so the app can update it when the preference changes.

Retrieve. When you ask for dinner ideas in a new chat, the app selects relevant information. It might use a category filter, a text search, or a similarity search over embeddings. See Day 3 for how embeddings support search.

Use. The app includes the selected preference alongside the new question. The model generates a reply based on the information it receives.

Retrieval can miss even when the information exists. A filter might exclude a record or a search might rank it below the available slots. A retrieved detail can also be omitted during context assembly or overlooked in the generated answer.

An app can retrieve relevant information at reply time without retraining the model. Retrieval-augmented generation uses the same broad pattern with external evidence. Personal preferences and document evidence have different access and retention needs.

Worked example: two dinner requests

First chat. You say, “I’m vegetarian. Please skip mushrooms.” For this example, the app stores only the vegetarian preference.

New chat. Your visible message is “Suggest a quick dinner.” The app retrieves the vegetarian entry. The model input might contain an instruction to give a short recipe, the retrieved preference “prefers vegetarian meals,” and your new message.

Possible reply: “Try veggie tacos with black beans and avocado.” Mushroom risotto is also compatible with the information supplied. The request to skip mushrooms was absent from this input.

Now write “Suggest a quick vegetarian dinner without mushrooms.” Both preferences appear in the current question. The model can use both, and the app can check whether the suggested ingredients satisfy them.

To debug a missed preference, trace three separate questions. Was it saved? Was it selected and included in the context? Did the answer respect it? Each failure calls for a different fix.

Context, memory, KV cache and fine-tuning

Concept	What it does	Dinner example
Context window	Limits tokens available to the model at once.	The current question and selected notes fit here.
Saved memory	Stores information the app can retrieve for a later reply.	The vegetarian preference persists across chats.
KV cache	Reuses intermediate attention computations.	Speeds up processing during generation.
Fine-tuning	Updates learned model or adapter parameters.	Training examples can teach a consistent recipe format.

The context window limits how much tokenised information the model can handle at once. Saving more preferences does not expand that window.

Day 5’s KV cache stores intermediate attention tensors so inference can reuse computation. An app’s saved preference is data that it can select for future replies.

Fine-tuning updates model or adapter parameters using training examples. It is useful for changing patterns such as response format or tone. A user preference that changes next week is easier to edit in a separate memory record.

Glossary. A token is a unit processed by a model. Retrieval selects relevant stored information. Context assembly chooses and formats what reaches the model. Provenance records where a memory came from.

Designing memory people can trust

Store preferences at the right scope. “Skip mushrooms tonight” can be a one-off request. “I never eat mushrooms” may be a lasting preference. Consider the wording and source before saving it.

Handle updates explicitly. If someone now asks for a vegan dinner, follow their current request. The app can distinguish a one-time exception from a request to update a lasting preference.

Use an access filter before ranking stored records. Restrict retrieval to the correct user or workspace, and treat retrieved text as context data. Avoid sending irrelevant private details into the prompt.

Make stored information inspectable and editable where the product supports it. Define how deletion affects the primary record, search index and caches. Test that a deleted item stops appearing in retrieval.

Measure the stages independently. Check whether the correct preference was stored, whether retrieval found it, whether it survived context assembly, and whether the answer followed it. Include tests for missing, stale and conflicting preferences.

Mini quiz

1. What is on the desk in the analogy?

2. What is in the drawer?

3. Why might an old turn be missing?

4. What follows retrieval?

5. Why could the model suggest mushroom risotto?

6. Does saved memory require a parameter update?

7. What does the KV cache reuse?

8. How can you make a constraint available immediately?

Answers

1. The information available in the current model context.

2. Saved information the app can select for later use.

3. The app may omit or summarise it when assembling context.

4. The app includes selected information in the model input.

5. The no-mushrooms constraint may be absent or overlooked.

6. No. The app can include the preference at reply time.

7. Intermediate attention keys and values.

8. Include it explicitly in the current request.

Interview practice

1. What is the difference between history and context?

History is the conversation record. Context is the input actually available to the model for a particular reply.

2. How can a new chat use a preference?

The app retrieves an eligible saved record and includes it with the current request.

3. Can saving a preference guarantee the next answer uses it?

Check retrieval, context assembly and adherence separately. Saving covers only the first stage.

4. Why summarise older messages?

Summaries reduce token use. A summary can drop constraints, so preserve important details explicitly.

5. Where does semantic search fit?

It can rank candidate memories by relevance. Identity filtering and scope checks still need to apply.

6. What would you store for a preference?

A value, owner, scope, source, timestamp, and any expiry or supersession information.

Interview practice, continued

7. How would you resolve conflicting preferences?

Use the latest explicit request for this turn. Distinguish a temporary override from a durable update.

8. What metrics would you track?

Eligible memory recall, irrelevant retrieval rate, context inclusion, answer adherence, and latency.

9. How would you test isolation?

Create similar preferences under two test accounts and verify that retrieval always returns only eligible records.

10. How would you test deletion?

Remove a test preference, query again, and verify it is absent from retrieval, derived indexes and application caches.

11. How does a KV cache differ from persistent memory?

The cache is intermediate model state for efficient inference. Persistent memory is application data available to future requests.

12. When would you choose fine-tuning?

When you want recurring behaviour learned from examples, such as consistent formatting. Use editable records for changing user preferences.

Sources and further reading

OpenAI Memory FAQ, one product example
https://help.openai.com/en/articles/8590148-memory-faq

Hugging Face Transformers, How caching works
https://huggingface.co/docs/transformers/en/cache_explanation

Lewis et al., Retrieval-Augmented Generation
https://arxiv.org/abs/2005.11401

Hu et al., LoRA
https://arxiv.org/abs/2106.09685

The dinner example illustrates one way an app could implement memory. Consult the relevant product documentation for current controls.